



K.N. Toosi University

Aidin Sahneh

Student ID: 40120243

Yazdan Bayat

|Amirali Rouhi

Instructor: Dr. Aliyari

Course: Artificial Intelligence

Q2

2.1.1 Dataset Overview

The data employed in this study was retrieved through the OpenWeather API, covering four specific locations in Mauritius: Moka, Curepipe, Vacoas, and Quatre Bornes. The data was gathered between May 1st and May 31st, 2021, with four measurements taken each hour, amounting to 2,976 entries per site.

The six recorded weather features include:

- Air Temperature (°C)
- Wind Velocity (m/s)
- Wind Bearing (°)
- Atmospheric Pressure (Pa)
- Relative Humidity (%)
- Cloud Coverage (%)

Information was gathered via edge computing devices such as desktop and mobile applications, which queried the OpenWeather API every 15 minutes. The data was saved both locally in a MySQL database and remotely on IBM Cloud's Cloudant (NoSQL) platform. This dual-storage approach enabled both local and cloud-based real-time data analysis and forecasting.

2.1.2 Filtering Weather Data for Cities in France

This phase involves narrowing down the dataset to retain only meteorological records related to French cities from the given file `weather_prediction_dataset.csv`. The file includes data for various urban locations, with each feature labeled in the format `CITYNAME_feature`. As the goal is to replicate spatial grouping as done in the original research, it's necessary to discard data unrelated to France and focus exclusively on French locations.

To perform this operation, we begin by determining which cities in the dataset belong to France. After identifying them, we extract the corresponding feature columns while maintaining the time identifiers (`DATE` and `MONTH`). The outcome is a refined dataset limited to weather metrics from French cities.

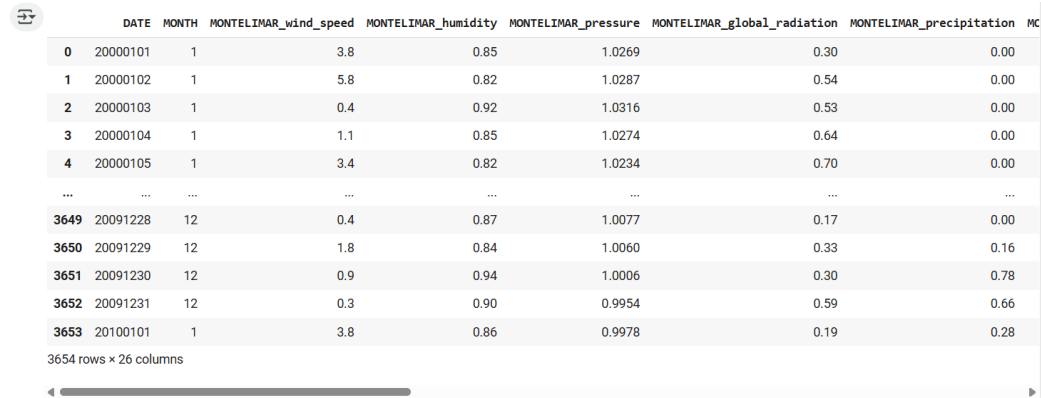
The Python script below (executed in Google Colab) demonstrates the filtering process:

```

1 import pandas as pd
2
3 # Load original dataset
4 df = pd.read_csv('weather_prediction_dataset.csv')
5
6 # Define French city names based on prefix examination
7 french_cities = ['MONTELMAR', 'PERPIGNAN', 'TOURS']
8
9 # Time columns to keep
10 base_columns = ['DATE', 'MONTH']
11
12 # Gather all relevant columns
13 french_columns = base_columns.copy()
14 for col in df.columns:
15     if any(col.startswith(city + '_') for city in french_cities):
16         french_columns.append(col)
17
18 # Extract the refined DataFrame
19 df_french = df[french_columns]

```

The final `df_french` DataFrame includes 3654 entries and only the weather attributes pertaining to selected French cities.



	DATE	MONTH	MONTELMAR_wind_speed	MONTELMAR_humidity	MONTELMAR_pressure	MONTELMAR_global_radiation	MONTELMAR_precipitation	MC
0	20000101	1	3.8	0.85	1.0269	0.30	0.00	
1	20000102	1	5.8	0.82	1.0287	0.54	0.00	
2	20000103	1	0.4	0.92	1.0316	0.53	0.00	
3	20000104	1	1.1	0.85	1.0274	0.64	0.00	
4	20000105	1	3.4	0.82	1.0234	0.70	0.00	
...	...	...	...	...	...	...	...	...
3649	20091228	12	0.4	0.87	1.0077	0.17	0.00	
3650	20091229	12	1.8	0.84	1.0060	0.33	0.16	
3651	20091230	12	0.9	0.94	1.0006	0.30	0.78	
3652	20091231	12	0.3	0.90	0.9954	0.59	0.66	
3653	20100101	1	3.8	0.86	0.9978	0.19	0.28	

3654 rows x 26 columns

Figure 1: Filtered dataset containing only weather data for French cities.

2.1.3 Data Preparation

In this work, we implement a preprocessing strategy influenced by the publication titled *A Real-Time Collaborative Machine Learning Based Weather Forecasting System with Multiple Predictor Locations*. The dataset consists of daily climate data from three French cities: TOURS, PERPIGNAN, and MONTELMAR. The objective is to reformat this time-series data for effective use in training predictive

models, particularly to estimate a weather-related value (e.g., temperature) at a future timestamp based on previous observations.

The preparation workflow consists of the following phases:

1. **Column Selection:** We isolate the relevant attributes for the three French regions.
2. **Feature Scaling:** We apply Min-Max normalization to all numerical columns so their values lie within the interval $[0, 1]$. This step is important for improving the performance of models like neural networks and regression techniques.
3. **Sequence Windowing:** We segment the time-series into overlapping sequences. In particular, each training entry comprises 36 previous days as input, with the 37th day serving as the prediction target. This approach resembles a rolling window strategy and simulates the real-time forecasting scenario described in the referenced work.
4. **Tensor Formation:** The final processed data adopts a 3D format:
 - $X \in \mathbb{R}^{3618 \times 36 \times 24}$ — 3618 training samples, each consisting of 36 steps and 24 features
 - $y \in \mathbb{R}^{3618}$ — each element corresponds to the forecasted value (e.g., temperature at TOURS)

The code snippet below outlines the entire preparation process:

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import MinMaxScaler
4
5 # STEP 1: Separate temporal info from features
6 df_meta = df_french[['DATE', 'MONTH']]
7 df_features = df_french.drop(columns=['DATE', 'MONTH'])
8
9 # STEP 2: Normalize numerical values using Min-Max scaling
10 scaler = MinMaxScaler()
11 normalized_array = scaler.fit_transform(df_features)
12 df_normalized = pd.DataFrame(normalized_array, columns=
    df_features.columns)
13
14 # Reinstate DATE and MONTH for alignment
15 df_normalized['DATE'] = df_meta['DATE'].values
16 df_normalized['MONTH'] = df_meta['MONTH'].values
17
```

```

18 # STEP 3: Generate sequences using a sliding window method (size
    = 36)
19 def create_sliding_windows(dataframe, window_size=36,
    target_column=None):
20     X, y = [], []
21     for i in range(len(dataframe) - window_size):
22         window = dataframe.iloc[i:i+window_size].drop(columns=["DATE", "
            MONTH"]).values
23         X.append(window)
24         if target_column:
25             y.append(dataframe.iloc[i + window_size][target_column])
26         else:
27             y.append(dataframe.iloc[i + window_size].drop(labels=["DATE", "
                MONTH"]).values)
28     return np.array(X), np.array(y)
29
30 # STEP 4: Specify the output variable (e.g., TOURS_temp_mean)
31 target_variable = "TOURS_temp_mean"
32 X, y = create_sliding_windows(df_normalized, window_size=36,
    target_column=target_variable)
33
34 # STEP 5: Output dimensions for verification
35 print("X shape (samples, window, features):", X.shape)
36 print("y shape (samples,):", y.shape)

```

Listing 1: Python implementation for windowed dataset generation

Figure 2 displays the output confirming the dimensionality of the resulting dataset.



```

X shape (samples, window, features): (3618, 36, 24)
y shape (samples,): (3618,)

```

Figure 2: Shape of processed input — 3618 entries, 36 past steps, 24 attributes

2.1.4 Sliding Window Setup

In this stage, we restructure the French meteorological dataset for use in a supervised learning model. The target is to forecast the TOURS_wind_speed value by leveraging data from the prior 5 days (using a temporal sliding window method). The procedure is outlined below:

- Discard the DATE and MONTH fields from the feature matrix.
- Scale all numerical features using Min-Max normalization.

- Partition the data into training (years prior to 2009) and testing (year 2009).
- Construct rolling windows of size 5 from the input matrix, forming 3D arrays with dimensions $[samples, 5, features]$.
- Generate the corresponding target vectors y_{train} and y_{test} by using the TOURS_wind_speed entry immediately following each sequence.

The Python script below executes these preprocessing tasks:

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import MinMaxScaler
4
5 # Step 1: Remove temporal indicators
6 df_meta = df_french[['DATE', 'MONTH']]
7 df_features = df_french.drop(columns=['DATE', 'MONTH'])
8
9 # Step 2: Normalize feature values
10 scaler = MinMaxScaler()
11 normalized_array = scaler.fit_transform(df_features)
12 df_normalized = pd.DataFrame(normalized_array, columns=
    df_features.columns)
13 df_normalized['DATE'] = df_meta['DATE'].values
14 df_normalized['MONTH'] = df_meta['MONTH'].values
15
16 # Step 3: Initialization
17 window_size = 5
18 target_col = 'TOURS_wind_speed'
19
20 # Step 4: Convert DATE into datetime objects
21 df_normalized['DATE'] = pd.to_datetime(df_normalized['DATE'].
    astype(str), format='%Y%m%d')
22
23 # Step 5: Create training and testing subsets
24 train_df = df_normalized[df_normalized['DATE'].dt.year < 2009].
    reset_index(drop=True)
25 test_df = df_normalized[df_normalized['DATE'].dt.year == 2009].
    reset_index(drop=True)
26
27 # Step 6: Isolate input features
28 feature_columns = [col for col in df_normalized.columns if col
    not in ['DATE', 'MONTH', target_col]]
29 train_features = train_df[feature_columns]
30 test_features = test_df[feature_columns]
31
32 # Step 7: Construct temporal windows
33 def create_windowed_data(data, window_size):
34 X = []

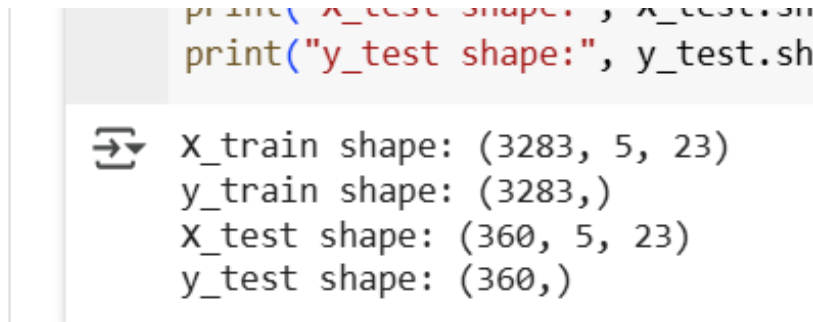
```

```

35 for i in range(window_size, len(data)):
36     X.append(data.iloc[i - window_size:i].values)
37     return np.array(X)
38
39 X_train = create_windowed_data(train_features, window_size)
40 X_test = create_windowed_data(test_features, window_size)
41
42 # Step 8: Prepare target arrays
43 y_train = train_df[target_col].iloc[window_size:].reset_index(
44     drop=True).values
45 y_test = test_df[target_col].iloc[window_size:].reset_index(drop
46     =True).values
47
48 # Step 9: Display dimensions
49 print("X_train shape:", X_train.shape)
50 print("y_train shape:", y_train.shape)
51 print("X_test shape:", X_test.shape)
52 print("y_test shape:", y_test.shape)

```

Listing 2: Sliding Window Setup for TOURS Wind Speed Prediction



```

print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.sh

```

X_train shape: (3283, 5, 23)
 y_train shape: (3283,)
 X_test shape: (360, 5, 23)
 y_test shape: (360,)

Figure 3: Visualization of rolling window structure for forecasting wind speed in TOURS. Each input consists of 5 days of consecutive data.

2.2.1 – Concept and Usage of Collaborative Machine Learning in the Paper

Collaborative Machine Learning refers to a learning paradigm where models are trained using data from multiple nearby regions or sources, rather than relying solely on data from a single location. This approach leverages the spatial correlation between neighboring areas to improve the overall predictive performance.

In the context of the paper, collaborative machine learning is applied by combining weather data from multiple geographically close locations to build a shared prediction model. Specifically, instead of training a model using only the weather

history of a single city, the authors include data from several surrounding cities. These combined inputs are used to predict the future weather condition of a target city.

The paper demonstrates that collaborative models achieve lower prediction error (MAPE) compared to non-collaborative models. Linear models such as Multiple Linear Regression (MLR) and Multiple Polynomial Regression (MPR) benefit the most from collaboration. This confirms that the inclusion of spatially diverse yet related data can enhance learning, especially in the context of weather forecasting.

2.3 Designing a Neural Network for Weather Prediction

In this section, we describe how to design a feedforward neural network to predict wind speed using weather data from the city of TOURS. We follow the structure shown in Figure 4, which includes:

- One input layer receiving a sequence of weather features.
- One hidden layer with ReLU activation.
- One output layer with a linear activation.
- A backpropagation training loop.

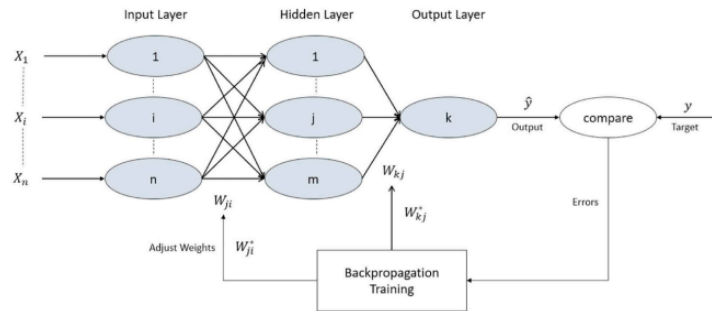


Figure 4: Neural Network Architecture

We prepare our data using a sliding window approach (window size 5), normalize it using Min-Max scaling, and split it into training (years before 2009) and testing (year 2009) subsets. We then build a neural network using TensorFlow.

Below is the Python code for designing and evaluating the neural network:

```
1 model = Sequential([
2 Dense(16, input_shape=(X_train_flat.shape[1],), activation='relu
  '),
3 Dense(1, activation='linear')
4 ])
5
6 optimizer = SGD(learning_rate=1e-3)
7 model.compile(optimizer=optimizer, loss='mse', metrics=['mae'])
8
9 history = model.fit(
10 X_train_flat, y_train,
11 validation_data=(X_test_flat, y_test),
12 epochs=200,
13 batch_size=32,
14 verbose=0
15 )
16 y_pred = model.predict(X_test_flat).flatten()
17
18 mse = mean_squared_error(y_test, y_pred)
19 mae = mean_absolute_error(y_test, y_pred)
20 r2 = r2_score(y_test, y_pred)
```

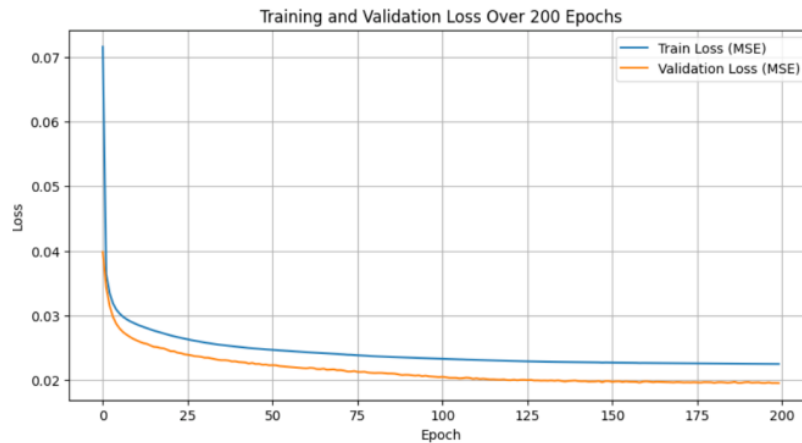


Figure 5: Training and Validation Loss over 200 Epochs

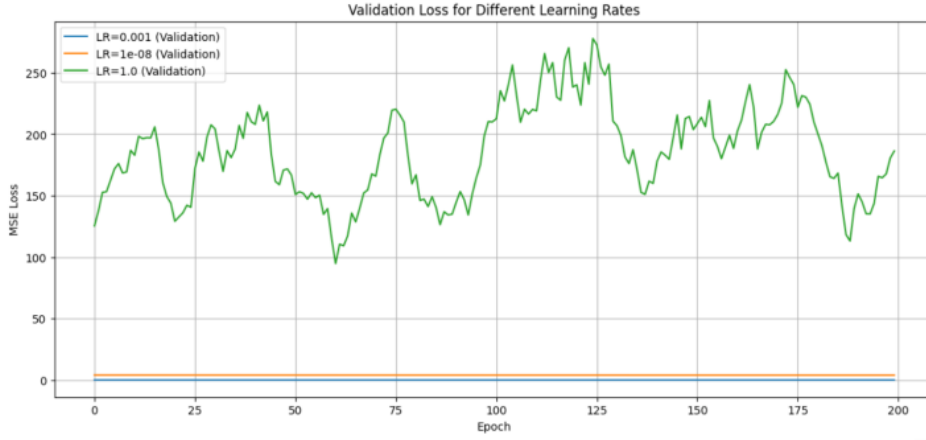


Figure 6: Validation Loss for Learning Rates 10^{-3} , 10^{-8} , and 10^0

0.1 Final Evaluation Metrics

	Learning Rate	MSE	MAE	R2 Score
0	1.000000e-03	0.025612	0.129027	-0.302205
1	1.000000e-08	0.050968	0.192995	-1.591397
2	1.000000e+00	81.979199	9.053150	-4167.126832

Figure 7: MSE, MAE, and R^2 for Different Learning Rates

Expanded Neural Architecture

To assess the impact of increased model complexity, we constructed a neural network with additional hidden layers. This aimed to investigate whether a deeper structure could enhance generalization ability and minimize validation error.

The revised model contains:

- Input Layer: A flattened input vector.
- Hidden Layer 1: 64 units, activated via ReLU.
- Hidden Layer 2: 32 units, ReLU activation.
- Hidden Layer 3: 16 units, ReLU activation.
- Output Layer: Single unit with linear activation function.

Training was conducted on the same dataset using stochastic gradient descent (SGD) with a learning rate of 10^{-3} and mean squared error (MSE) as the loss criterion, running for 200 training iterations.

Python Implementation

```
1 deep_model = Sequential([
2     Dense(64, input_shape=(X_train_flat.shape[1],), activation='relu
3     '),
4     Dense(32, activation='relu'),
5     Dense(16, activation='relu'),
6     Dense(1, activation='linear')
7 ])
8 optimizer = SGD(learning_rate=1e-3)
9 deep_model.compile(optimizer=optimizer, loss='mse', metrics=['
10     mae'])
11 history = deep_model.fit(
12     X_train_flat, y_train,
13     validation_data=(X_test_flat, y_test),
14     epochs=200,
15     batch_size=32,
16     verbose=1
17 )
18
19 y_pred = deep_model.predict(X_test_flat).flatten()
20
21 mse = mean_squared_error(y_test, y_pred)
22 mae = mean_absolute_error(y_test, y_pred)
23 r2 = r2_score(y_test, y_pred)
```

Loss Visualization

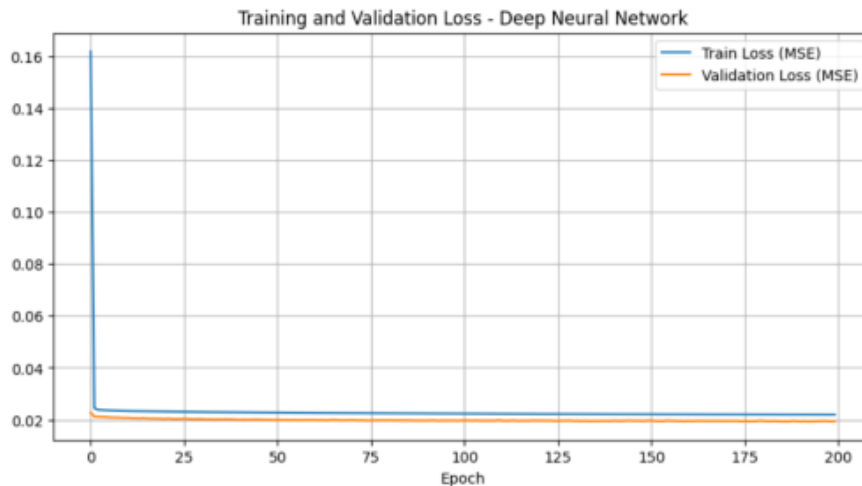


Figure 8: Training and Validation Loss for Deep Neural Network

Monitoring Weight Trajectories During Training

To gain insights into how the model adapts during learning, we observed the evolution of weights connected to a specific neuron within the first hidden layer. A custom Keras callback was utilized to record the incoming weights of neuron 0 in the initial hidden layer after each training cycle.

The neural structure employed includes three hidden layers and a single output node with linear activation. Training was executed across 200 epochs using the stochastic gradient descent (SGD) optimizer with a learning rate set to 0.01.

Python Code

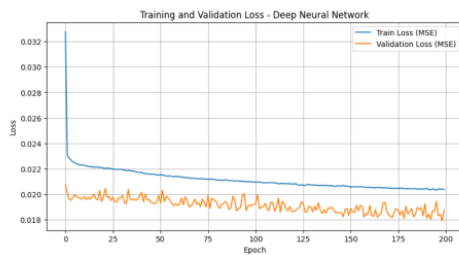
```
1 class WeightLogger(Callback):
2     def __init__(self, layer_index=0, neuron_index=0):
3         super().__init__()
4         self.layer_index = layer_index
5         self.neuron_index = neuron_index
6         self.history = []
7
8     def on_epoch_end(self, epoch, logs=None):
9         weights = self.model.layers[self.layer_index].get_weights()[0]
10        self.history.append(weights[:, self.neuron_index].copy())
11
12 model = Sequential([
```

```

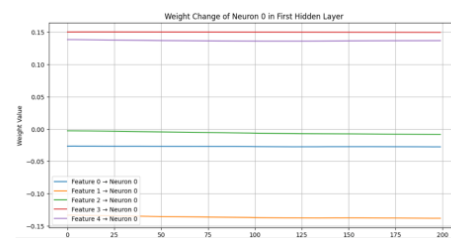
13 Dense(64, input_shape=(X_train_flat.shape[1],), activation='relu
    '),
14 Dense(32, activation='relu'),
15 Dense(16, activation='relu'),
16 Dense(1, activation='linear')
17 ])
18 optimizer = SGD(learning_rate=0.01)
19 model.compile(optimizer=optimizer, loss='mse', metrics=['mae'])
20
21 weight_logger = WeightLogger(layer_index=0, neuron_index=0)
22 history = model.fit(
23 X_train_flat, y_train,
24 validation_data=(X_test_flat, y_test),
25 epochs=200,
26 batch_size=32,
27 callbacks=[weight_logger]
28 )

```

Training and Weight Visualization



(a) Training and validation loss over 200 epochs.



(b) Weight change for neuron 0 in the first hidden layer.

Figure 9: Model performance and internal weight evolution during training.